

Лабораторная работа №1

Функции получения системной информации.

Цель работы: Получение практических навыков по программированию в Win32 API с использованием аппаратных и системных функций.

Задание для выполнения лабораторной работы 1.

Разработать программу, обеспечивающую получение следующей системной информации:

1. Имя компьютера, имя пользователя.
2. Пути к системным каталогам Windows.
3. Версия операционной системы.
4. Системные метрики (3 системные метрики по вариантам)
5. Системные параметры (3 системных параметра):
6. Системные цвета (определить цвет для символьных констант и изменить его на любой другой):

6	COLOR_BTNHIGHLIGHT, COLOR_INACTIVEBORDER, COLOR_WINDOW
6	DestroyCaret, GetEnvironmentVariable, GetSystemDefaultLangID, SetDoubleClickTime

Программный код:

```
#include <iostream>
#include <windows.h>

int main() {

    // 1. Получение имени компьютера и имени пользователя

    char computerName[MAX_COMPUTERNAME_LENGTH + 1];
    DWORD size = sizeof(computerName);
    GetComputerNameA(computerName, &size);

    std::cout << "Имя компьютера: " << computerName << std::endl;
```

```
char userName[MAX_PATH];
DWORD userNameSize = sizeof(userName);
GetUserNameA(userName, &userNameSize);

std::cout << "Имя пользователя: " << userName << std::endl;

// 2. Получение путей к системным каталогам Windows

char systemDirectory[MAX_PATH];
GetSystemDirectoryA(systemDirectory, MAX_PATH);

std::cout << "Путь к системному каталогу Windows: " << systemDirectory <<
std::endl;

char windowsDirectory[MAX_PATH];
GetWindowsDirectoryA(windowsDirectory, MAX_PATH);

std::cout << "Путь к каталогу Windows: " << windowsDirectory << std::endl;

// 3. Получение версии операционной системы

OSVERSIONINFOA osv;
ZeroMemory(&osv, sizeof(OSVERSIONINFOA));
osv.dwOSVersionInfoSize = sizeof(OSVERSIONINFOA);
GetVersionExA((LPOSVERSIONINFOA)&osv);

std::cout << "Версия операционной системы: " << osv.dwMajorVersion << "." <<
osv.dwMinorVersion << std::endl;

// 4. Получение системных метрик

int screenWidth = GetSystemMetrics(SM_CXSCREEN);
int screenHeight = GetSystemMetrics(SM_CYSCREEN);

std::cout << "Ширина экрана: " << screenWidth << " пикселей" << std::endl;
std::cout << "Высота экрана: " << screenHeight << " пикселей" << std::endl;

// 5. Получение системных параметров

int destroyCaret = DestroyCaret();
char environmentVariable[MAX_PATH];
DWORD environmentVariableSize = GetEnvironmentVariableA("PATH",
environmentVariable, MAX_PATH);
LANGID systemDefaultLangID = GetSystemDefaultLangID();
int doubleClickTime = GetDoubleClickTime();
```

```

std::cout << "Результат DestroyCaret: " << destroyCaret << std::endl;
std::cout << "Переменная окружения PATH: " << environmentVariable <<
std::endl;
std::cout << "Языковой идентификатор системы по умолчанию: " <<
systemDefaultLangID << std::endl;
std::cout << "Время между двойными кликами: " << doubleClickTime << "
миллисекунд" << std::endl;

// 6. Получение изначального цвета COLOR_BTNHIGHLIGHT

COLORREF btnHighlightColor = GetSysColor(COLOR_BTNHIGHLIGHT);
COLORREF inactiveBorderColor = GetSysColor(COLOR_INACTIVEBORDER);
COLORREF windowColor = GetSysColor(COLOR_WINDOW);

std::cout << "Цвет кнопок: " << btnHighlightColor << std::endl;
std::cout << "Цвет неактивных границ: " << inactiveBorderColor << std::endl;
std::cout << "Цвет окна: " << windowColor << std::endl;

return 0;
}

```

Вывод:

```

Имя компьютера: HP-D
Имя пользователя: diamo
Путь к системному каталогу windows: C:\WINDOWS\system32
Путь к каталогу Windows: C:\WINDOWS
Версия операционной системы: 10.0
Ширина экрана: 1536 пикселей
Высота экрана: 864 пикселей
Результат DestroyCaret: 0
Переменная окружения PATH:
Языковой идентификатор системы по умолчанию: 1049
Время между двойными кликами: 1000 миллисекунд
Цвет кнопок: 255
Цвет неактивных границ: 65280
Цвет окна: 16777215

```